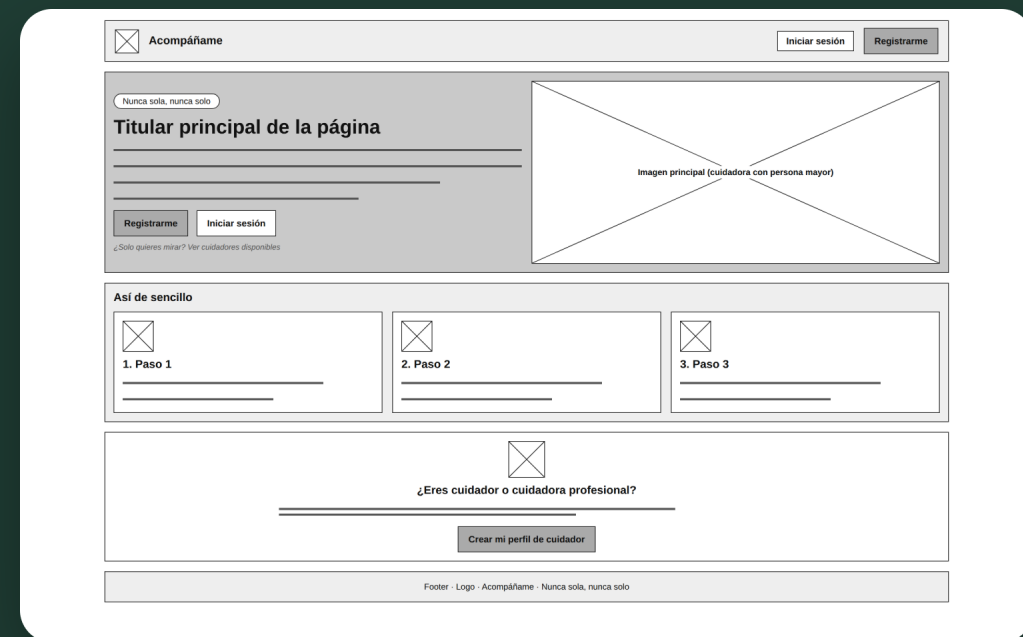


Proyecto final · Bootcamp Full Stack

Acompaña me

Cuidadores de confianza, cuando tu familia más lo necesita

Andrea Vallina González · Factoría F5 · Octubre 2026



EL PROBLEMA

Encontrar cuidado de confianza no debería ser tan difícil



Las familias

Buscan acompañamiento para un familiar mayor, en el hospital o en casa, y no saben a quién acudir.



Los cuidadores

Son profesionales sin un lugar donde publicar su perfil y recibir solicitudes.



El hueco

Llamadas interminables, papeleo y poca confianza entre unos y otros.

Nunca sola, nunca solo.

Un marketplace con dos roles

FAMILIA

- Busca cuidadores y consulta su perfil
- Envía una solicitud de servicio
- Sigue el estado de cada solicitud
- Paga el servicio aceptado
- Valora al cuidador al terminar

CUIDADOR

- Publica su perfil profesional
- Recibe solicitudes de familias
- Las acepta o las rechaza
- Aparece en las búsquedas con su valoración
- Consulta su historial de servicios

Todo en un único sitio, sin llamadas ni papeleo.

CÓMO ESTÁ CONSTRUIDO

Arquitectura en capas

Backend: Java 21 y Spring Boot

Controller → Service → Repository → Entity

Frontend: Vue 3 y Vite

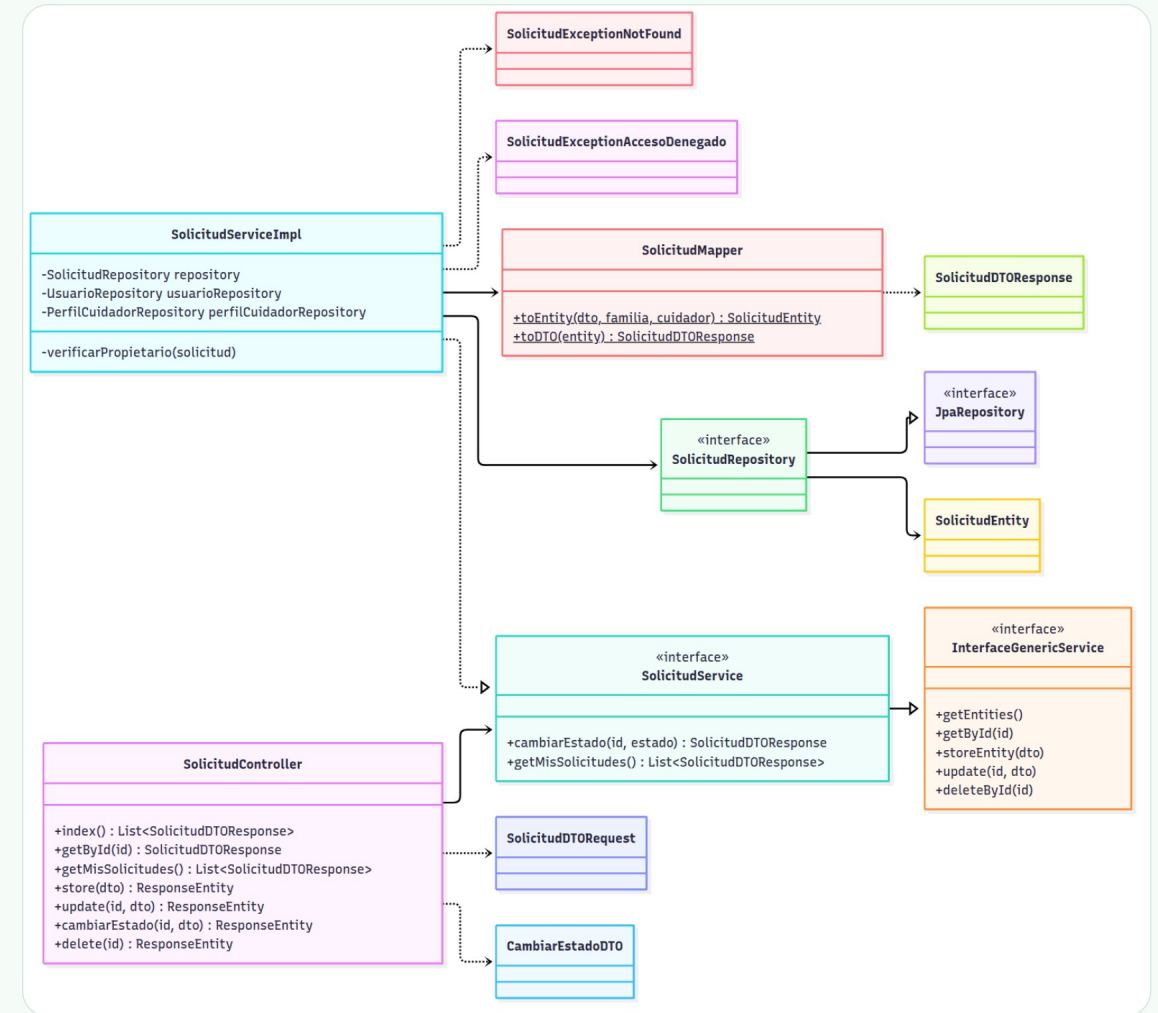
Pinia, Vue Router y patrón Repository

Base de datos: MySQL 8 en Docker

JPA / Hibernate

DTOs y Mappers

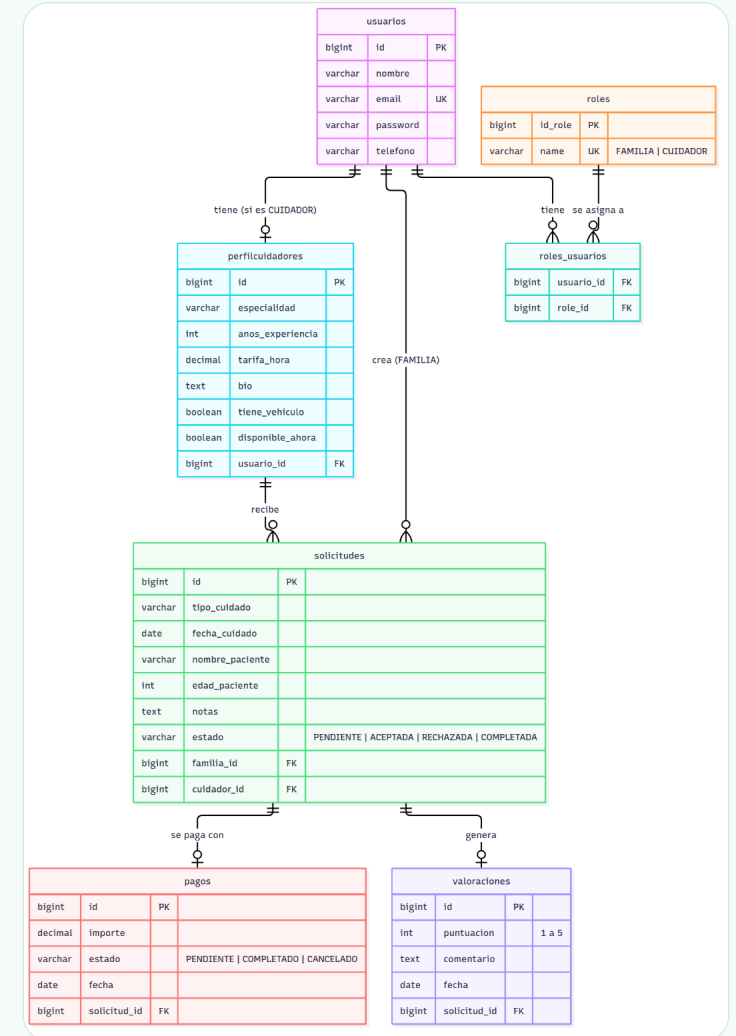
Nunca se expone una entidad JPA



CÓMO ESTÁ CONSTRUIDO

Modelo de datos

- Seis entidades: Usuario, Role, PerfilCuidador, Solicitud, Valoracion y Pago
- Los roles viven en una tabla propia con @ManyToMany, no en un enum
- Cada solicitud tiene su pago asociado, en relación uno a uno
- El pago pasa por tres estados: PENDIENTE, COMPLETADO y CANCELADO



Seguridad: cada usuario ve solo lo suyo

Autenticación

Spring Security con Basic Auth y contraseñas cifradas con BCrypt.

Autorización por rol

hasAuthority para FAMILIA y CUIDADOR en cada endpoint.

Protección IDOR

Cada usuario solo puede leer o modificar sus propios datos.

CORS controlado

Solo el frontend en localhost:5173 puede llamar a la API.

Siguiente paso: migrar de Basic Auth a JWT.

Del boceto al prototipo

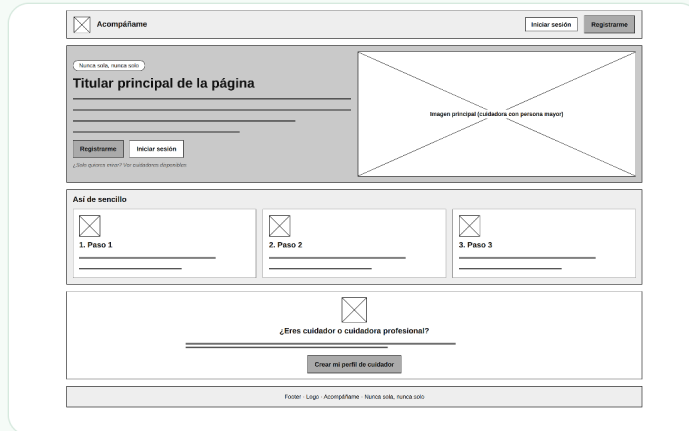
1 · Bocetos

2 · Wireframes

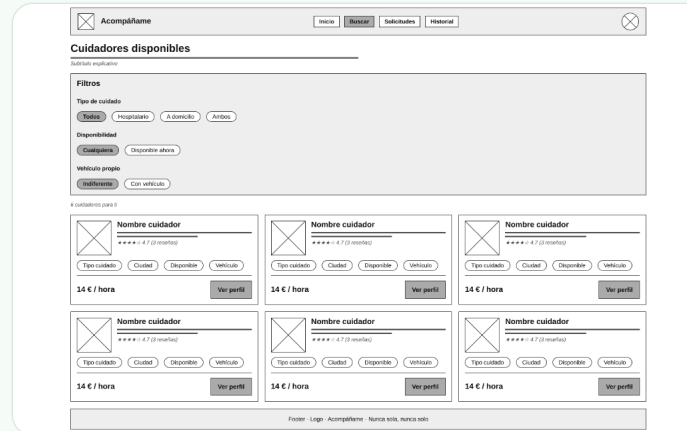
3 · Mockups

4 · Prototipo en Lovable

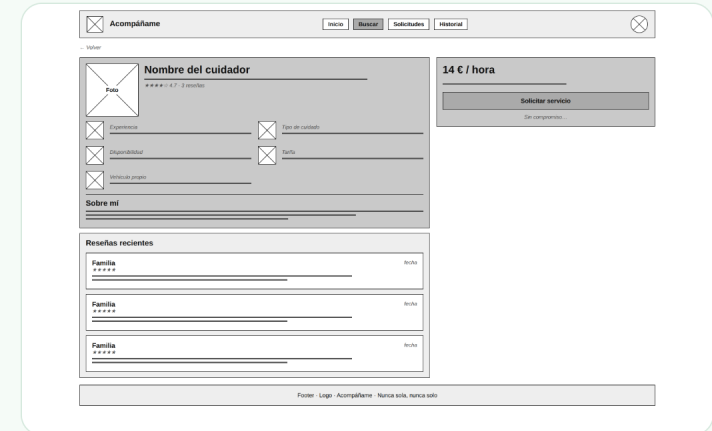
5 · Figma



Inicio



Búsqueda de cuidadores

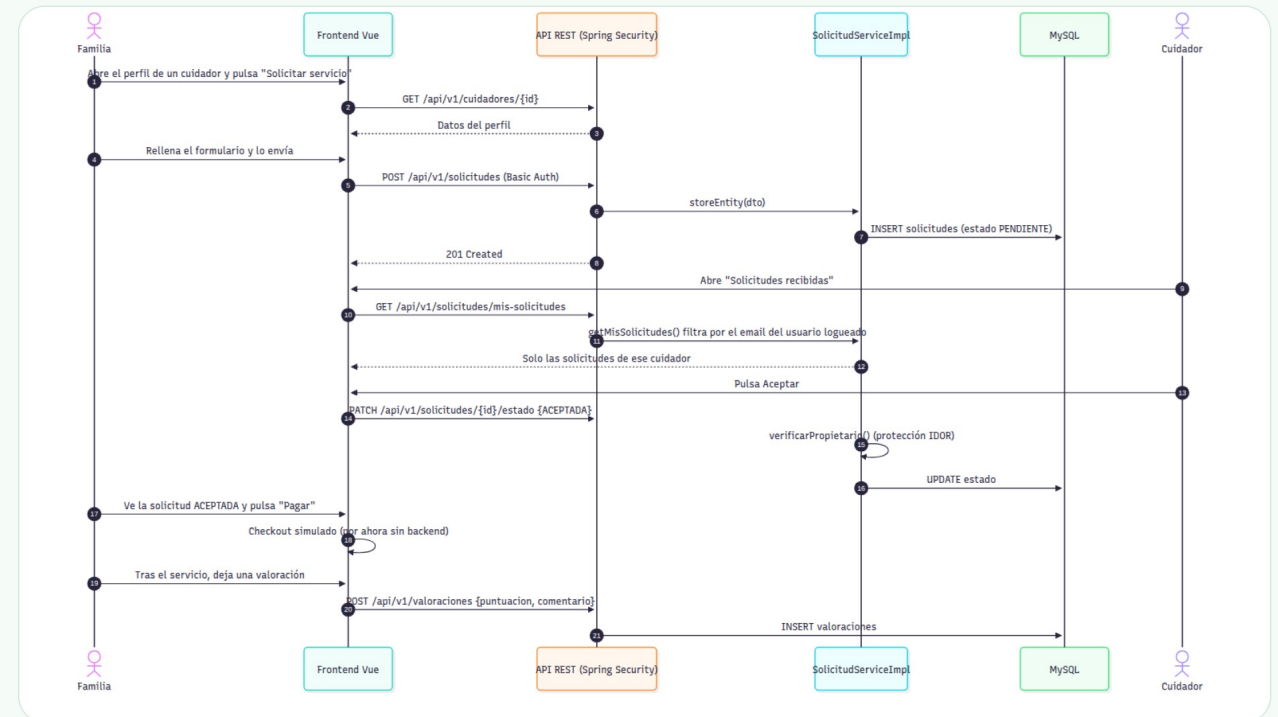


Perfil del cuidador

EL RECORRIDO

Así funciona una solicitud

1. La familia busca y elige un cuidador
2. Envía la solicitud de servicio
3. El cuidador la acepta o la rechaza
4. Si la acepta, se genera el pago
5. La familia paga y valora el servicio



CÓMO ESTÁ CONSTRUIDO

Frontend en Vue 3

- Vue 3 con script setup y Vite
- Pinia guarda la sesión en sessionStorage
- Vue Router con rutas según el rol
- Patrón Repository: las vistas nunca llaman a fetch
- CSS con BEM y variables, adaptable desde 768 px

13

pantallas

27

tests de componentes

Probar antes de entregar



Backend

147 tests

Cobertura del 90 % con JaCoCo.
Mockito en los servicios,
MockMvc y Testcontainers con
MySQL real en Docker.



Frontend

63 + 9 tests

63 unitarios con Vitest (79
% de cobertura) y 9 e2e con
Playwright que recorren la
app completa.



Casos límite

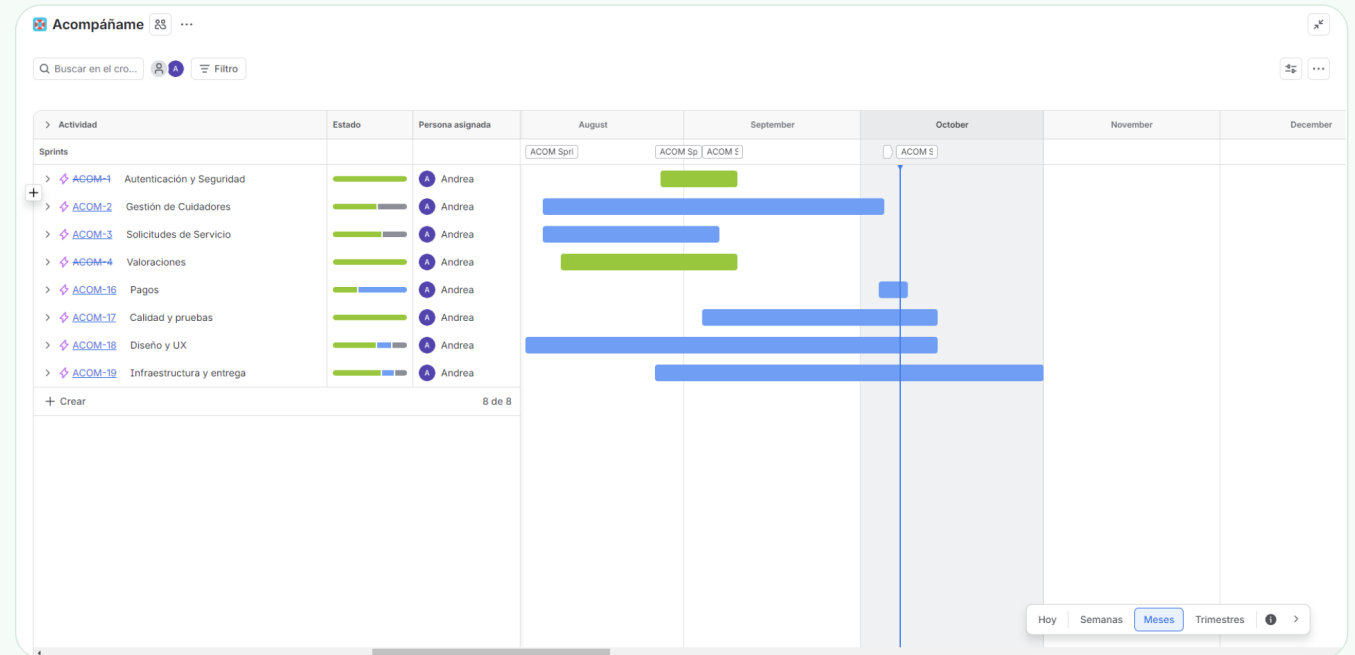
Lo difícil

Errores 400, 401, 403, 404
y 409, y acceso denegado
entre usuarios.

Cobertura por encima del 70 % pedido. Practiqué TDD en una vista: test en rojo, arreglo y refactorizar.

Scrum con JIRA

- 8 épicas
- 19 historias de usuario, con criterios en formato Gherkin
- 16 tareas técnicas
- 5 sprints, de agosto a octubre



Qué decidí y por qué

Decisión	Por qué
Patrón Repository en el frontend	Las vistas no conocen fetch ni las cabeceras
Roles en una tabla, no en un enum	Permite @ManyToMany y es más flexible
hasAuthority en vez de hasRole	Evita el prefijo ROLE_ que Spring añade solo
DTOs y Mappers siempre	Nunca se expone una entidad JPA
BEM y variables CSS	Un único sitio para colores y espaciados
Checkout simulado	Muestra el flujo de pago sin pasarela real

Qué mejoraría con más tiempo

Próximos pasos

- JWT en lugar de Basic Auth
- Conectar el checkout con el endpoint de pagos y, más adelante, con una pasarela real
- Alinear todas las pantallas con el prototipo de Lovable, incluida la edición de perfil
- Desplegar frontend y backend

Fase 2

- Chat en tiempo real
- Geolocalización de cuidadores
- Videollamadas
- Recordatorios y notificaciones
- Informes en PDF

Gracias

¿Preguntas?

github.com/AndreaVaGo/acompaname-backend

github.com/AndreaVaGo/acompaname-frontend

[linkedin.com/in/andrea-vallina-gonzalez](https://www.linkedin.com/in/andrea-vallina-gonzalez)

The screenshot displays the 'Acompáñame' web application interface. At the top, there is a navigation bar with the application name 'Acompáñame' on the left and a set of navigation links: 'Inicio', 'Buscar', 'Solicitudes', and 'Historial' on the right. Below the navigation bar, the main heading is 'Solicitudes recibidas'. Underneath this heading, there is a sub-heading 'Responde a las familias'. The interface is divided into two columns, each containing a form for processing a request. Each form has a title 'Nombre familia / cuidador' and a 'Estado' button. Below the title, there are two input fields, each preceded by a placeholder 'Icono'. Further down, there is a section labeled 'Información sobre el paciente' with a text input field. At the bottom of each form, there are two buttons: 'Aceptar' (Accept) and 'Rechazar' (Reject). A footer at the very bottom of the page reads 'Footer - Logo - Acompáñame - Nunca sola, nunca solo'.